Frameworks Modernos e estrutura do Spring Boot e Spring MVC

Objetivos da aula

- Nesta aula vamos compreender o Spring Framework e sua evolução para o Spring Boot dentro do contexto de arquiteturas back-end modernas.
- Você entenderá como o Spring MVC implementa o padrão MVC e como conceitos como Inversão de Controle e Injeção de Dependências estruturam aplicações profissionais.
- Ao final, será capaz de explicar teoricamente como esses elementos se conectam para formar a base arquitetural do framework.

O que é o Spring Framework

- O Spring Framework é um framework open-source para desenvolvimento de aplicações Java.
- Ele foi criado para simplificar a construção de sistemas corporativos complexos.
- Seu foco sempre esteve na modularidade, flexibilidade e organização arquitetural.

O Contexto de Surgimento do Spring

- No início dos anos 2000, o desenvolvimento Java corporativo era fortemente dependente do EJB.
- Esse modelo era considerado complexo e excessivamente verboso.
- O Spring surgiu como uma alternativa mais leve e desacoplada.

Filosofia do Spring

- O Spring foi projetado com base em princípios como baixo acoplamento e alta coesão.
- Ele promove a separação clara de responsabilidades dentro da aplicação.
- Seu objetivo é permitir que o desenvolvedor foque na regra de negócio.

Evolução para o Spring Boot

- Com o tempo, o Spring tornou-se extremamente poderoso, mas também complexo em configuração.
- O Spring Boot surgiu para simplificar esse processo.
- Ele automatiza configurações e reduz a necessidade de arquivos extensos.

O que é o Spring Boot

- O Spring Boot é uma extensão do Spring Framework.
- Ele facilita a criação de aplicações prontas para produção.
- Seu principal objetivo é reduzir configuração manual e acelerar desenvolvimento.

Configuração Automática

- Um dos pilares do Spring Boot é a auto-configuration.
- O framework detecta dependências no projeto e configura automaticamente o necessário.
- Isso reduz esforço e padroniza estrutura.

Starter Dependencies

- O Spring Boot utiliza o conceito de “starters”.
- Eles agrupam dependências comuns para facilitar configuração.
- Exemplos:
  - spring-boot-starter-web
  - spring-boot-starter-data-jpa
  - spring-boot-starter-security

Esses pacotes organizam o ecossistema do projeto.

Arquitetura Base do Spring

- O Spring é organizado em módulos independentes.
- Cada módulo resolve um tipo específico de problema.
- Essa modularidade permite que o desenvolvedor utilize apenas o necessário.

Spring MVC

- O Spring MVC é o módulo responsável por implementar o padrão Model–View–Controller.
- Ele organiza o fluxo de requisições HTTP dentro da aplicação.
- É a base das APIs REST construídas com Spring.

O Papel do DispatcherServlet

- No Spring MVC, o DispatcherServlet é o ponto central de entrada.
- Ele recebe todas as requisições HTTP.
- Depois, direciona cada requisição ao controlador adequado.

Fluxo de Requisição no Spring MVC

- Requisição HTTP chega ao DispatcherServlet.
- O controlador apropriado é identificado.
- O método correspondente é executado.
- A resposta é construída e retornada ao cliente.

Esse fluxo implementa o padrão MVC de forma estruturada.

Controllers no Spring

- Controllers são classes responsáveis por lidar com requisições.
- Eles definem endpoints e direcionam chamadas para a lógica de negócio.
- Não devem conter regras complexas.

O Conceito de Bean

- No Spring, um Bean é um objeto gerenciado pelo container do framework.
- O ciclo de vida desses objetos é controlado pelo Spring.
- Isso permite gerenciamento centralizado de dependências.

Container do Spring

- O container é responsável por instanciar e gerenciar Beans.
- Ele controla dependências e ciclo de vida.
- Esse mecanismo é central na arquitetura do framework.

Inversão de Controle (IoC)

- Inversão de Controle significa que o framework controla a criação e gestão dos objetos.
- O desenvolvedor não instancia manualmente suas dependências.
- O controle da aplicação é invertido para o container.

Injeção de Dependências

- Injeção de Dependências é a implementação prática da IoC.
- O container fornece automaticamente os objetos necessários para cada classe.
- Isso reduz acoplamento entre componentes.

Benefícios da IoC

- Redução de acoplamento
- Maior testabilidade
- Facilidade de manutenção
- Maior modularidade

Esses benefícios são centrais na arquitetura moderna.

Spring e Arquitetura em Camadas

- O Spring facilita a organização em camadas:
  - Controller
  - Service
  - Repository

Cada camada possui papel bem definido dentro da aplicação.

Camada Service no Spring

- A camada Service concentra regras de negócio.
- Ela é composta por Beans gerenciados pelo container.
- Recebe chamadas dos Controllers e interage com Repositories.

Camada Repository

- A camada Repository lida com persistência de dados.
- O Spring integra facilmente com JPA e bancos relacionais.
- Ele abstrai operações complexas de banco.

Integração com Banco de Dados

- O Spring Boot facilita integração com bancos como:
  - PostgreSQL
  - MySQL
  - Oracle Database

Essa integração é configurada de forma simplificada.

Spring e REST

- O Spring MVC é amplamente utilizado para construção de APIs REST.
- Ele permite mapear métodos HTTP para métodos Java.
- Isso facilita desenvolvimento de serviços web modernos.

JSON como Padrão

- O retorno padrão em APIs Spring é JSON.
- A conversão de objetos Java para JSON é automatizada.
- Isso simplifica comunicação com front-end.

Segurança no Spring

- O ecossistema Spring inclui o módulo Spring Security.
- Ele oferece autenticação e autorização robustas.
- É amplamente utilizado em aplicações corporativas.

Spring em Grandes Empresas

- Diversas empresas utilizam Spring em produção.
  - Netflix
  - Amazon
  - Alibaba

- Isso demonstra sua relevância no mercado.

Spring e Microsserviços

- O Spring Boot é muito utilizado na construção de microsserviços.
- Ele facilita criação de aplicações independentes.
- Integra-se facilmente com orquestradores como Kubernetes.

Escalabilidade

- Aplicações Spring são projetadas para ambientes escaláveis.
- Elas podem ser distribuídas em múltiplas instâncias.
- Isso atende demandas de sistemas de alta carga.

Padronização Arquitetural

- O Spring impõe uma estrutura organizada.
- Ele incentiva boas práticas desde o início.
- Isso reduz improvisação arquitetural.

Modularidade

- O ecossistema Spring é composto por diversos projetos independentes.
- Cada módulo resolve um tipo específico de problema.
- Essa modularidade permite flexibilidade arquitetural.

Produtividade

- Spring Boot reduz configuração inicial.
- Ele permite iniciar projetos rapidamente.
- Isso aumenta produtividade das equipes.

Convenção sobre Configuração

- O Spring Boot adota o princípio de convenção sobre configuração.
- Ele define padrões padrão para evitar decisões repetitivas.
- O desenvolvedor configura apenas o que é necessário.

Arquitetura Orientada a Componentes

- Spring promove aplicações compostas por componentes desacoplados.
- Cada componente possui responsabilidade específica.
- O container gerencia as conexões entre eles.

Spring como Base Corporativa

- O Spring tornou-se padrão em muitas aplicações empresariais.
- Ele é amplamente adotado no mercado financeiro e corporativo.
- Sua maturidade o torna confiável para sistemas críticos.

Curva de Aprendizado

- Embora poderoso, o Spring possui curva de aprendizado relevante.
- Compreender IoC e Beans exige entendimento arquitetural.
- Essa complexidade é compensada por robustez.

Comparação com Frameworks Leves

- Frameworks leves exigem menos configuração inicial.
- Spring oferece mais estrutura e robustez.
- A escolha depende do contexto do projeto.

Papel Teórico do Spring na Formação

- Estudar Spring não é apenas aprender uma ferramenta.
- É compreender conceitos arquiteturais aplicados.
- Ele materializa padrões como MVC e IoC.

Conexão com Arquitetura Moderna

- Spring Boot conecta MVC, IoC, REST e microsserviços.
- Ele integra diversos conceitos em uma estrutura coesa.
- Isso o torna uma base sólida para aplicações modernas.

Síntese da Estrutura

- Spring Framework → Base conceitual
- Spring MVC → Implementação do padrão MVC
- Spring Boot → Simplificação e automação

Esses três elementos estruturam aplicações back-end profissionais.

Encerramento

- Você aprendeu:
  - O que é o Spring Framework
  - Como surgiu o Spring Boot
  - Como o Spring MVC implementa o padrão MVC
  - O papel dos Beans, IoC e Injeção de Dependências
  - Por que o Spring é amplamente adotado no mercado

Agora você compreende teoricamente como o Spring estrutura aplicações modernas e fundamenta arquiteturas back-end corporativas.

PUC Minas

© **PUC Minas** • Todos os direitos reservados, de acordo com o art. 184 do Código Penal e com a lei 9.610 de 19 de fevereiro de 1998.
Proibidas a reprodução, a distribuição, a difusão, a execução pública, a locação e quaisquer outras
modalidades de utilização sem a devida autorização da Pontifícia Universidade Católica de Minas Gerais.